1.

Step 1: Prepare Evidence

```bash
# Create sample evidence
echo "Confidential forensic evidence" > evidence.txt
```

```
┌──(kali㉿kali)-[~/Semester_6]
└─$ mkdir Lab1_Advanced_Disk_Imaging

┌──(kali㉿kali)-[~/Semester_6]
└─$ ls
Lab1_Advanced_Disk_Imaging

┌──(kali㉿kali)-[~/Semester_6]
└─$ cd Lab1_Advanced_Disk_Imaging

┌──(kali㉿kali)-[~/Semester_6/Lab1_Advanced_Disk_Imaging]
└─$ # Create sample evidence
echo "Confidential forensic evidence" > evidence.txt

┌──(kali㉿kali)-[~/Semester_6/Lab1_Advanced_Disk_Imaging]
└─$ ls
evidence.txt

┌──(kali㉿kali)-[~/Semester_6/Lab1_Advanced_Disk_Imaging]
└─$ cat evidence.txt
Confidential forensic evidence

┌──(kali㉿kali)-[~/Semester_6/Lab1_Advanced_Disk_Imaging]
└─$ ls
evidence.txt
```

Step 2: Create Forensic Image Using dd

```bash
# Create a bit-by-bit copy
dd if=evidence.txt of=evidence.img bs=4K status=progress
```

Explanation:

- if = input file
- of = output image file
- bs = block size (4K recommended)
- status=progress shows progress

```
┌──(kali㉿kali)-[~/Semester_6/Lab1_Advanced_Disk_Imaging]
└─$ dd if=evidence.txt of=evidence.img bs=4K  conv=noerror,sync  status=progress
0+1 records in
1+0 records out
4096 bytes (4.1 kB, 4.0 KiB) copied, 6.5286e-05 s, 62.7 MB/s

┌──(kali㉿kali)-[~/Semester_6/Lab1_Advanced_Disk_Imaging]
└─$ ls
evidence.img  evidence.txt

┌──(kali㉿kali)-[~/Semester_6/Lab1_Advanced_Disk_Imaging]
└─$ ls
evidence.img  evidence.txt
```

Step 3: Verify Integrity

```bash
# SHA-256 hash
sha256sum evidence.txt
sha256sum evidence.img

# MD5 hash (optional)
md5sum evidence.txt
md5sum evidence.img
```

Expected Result:

- Hash values match → evidence not altered.
- Hash values differ → data tampered.

Sha256 Hash

```
┌──(kali㉿kali)-[~/Semester_6/Lab1_Advanced_Disk_Imaging]
└─$ sha256sum evidence.txt > sha256_hash.txt

┌──(kali㉿kali)-[~/Semester_6/Lab1_Advanced_Disk_Imaging]
└─$ sha256sum evidence.img >> sha256_hash.txt

┌──(kali㉿kali)-[~/Semester_6/Lab1_Advanced_Disk_Imaging]
└─$ ls
evidence.img  evidence.txt  sha256_hash.txt

┌──(kali㉿kali)-[~/Semester_6/Lab1_Advanced_Disk_Imaging]
└─$ cat sha256_hash.txt
516fc8a3fedace63daa40cf6841ea2c1c72cfdfd6be8c10819fef2d08663a45d  evidence.txt
516fc8a3fedace63daa40cf6841ea2c1c72cfdfd6be8c10819fef2d08663a45d  evidence.img
```

Md5 hash

```
┌──(kali㉿kali)-[~/Semester_6/Lab1_Advanced_Disk_Imaging]
└─$ md5sum evidence.txt > md5_hash.txt

┌──(kali㉿kali)-[~/Semester_6/Lab1_Advanced_Disk_Imaging]
└─$ md5sum evidence.img >> md5_hash.txt

┌──(kali㉿kali)-[~/Semester_6/Lab1_Advanced_Disk_Imaging]
└─$ ls
evidence.img  evidence.txt  md5_hash.txt  sha256_hash.txt

┌──(kali㉿kali)-[~/Semester_6/Lab1_Advanced_Disk_Imaging]
└─$ cat md5_hash.txt
83683609f68773a69de8e42f22eec099  evidence.txt
83683609f68773a69de8e42f22eec099  evidence.img

┌──(kali㉿kali)-[~/Semester_6/Lab1_Advanced_Disk_Imaging]
└─$
```

Resukt:

The SHA256 and MD5 hashes of the original file and the disk image were identical, confirming the integrity and authenticity of the forensic image. No alteration occurred during acquisition.

Step 4: Tampering Simulation

```bash
# Modify original evidence
echo "Malicious change" >> evidence.txt

# Check hash again
sha256sum evidence.txt
sha256sum evidence.img
```

- Shows how **hash functions detect tampering**.

Step 5: Verification

1. (Hex Viewer)

```bash
xxd evidence.txt | head
xxd evidence.img | head
```

- Compare the content visually.

```
┌──(kali㉿kali)-[~/Semester_6/Lab1_Advanced_Disk_Imaging]
└─$ xxd evidence.txt | head
xxd evidence.img | head
00000000: 436f 6e66 6964 656e 7469 616c 2066 6f72  Confidential for
00000010: 656e 7369 6320 6576 6964 656e 6365 0a    ensic evidence.
00000000: 436f 6e66 6964 656e 7469 616c 2066 6f72  Confidential for
00000010: 656e 7369 6320 6576 6964 656e 6365 0a    ensic evidence.

┌──(kali㉿kali)-[~/Semester_6/Lab1_Advanced_Disk_Imaging]
└─$
```

Explaination:

🔷 **xxd evidence.txt | head**

This command is used to:

1. **Convert the file into hexadecimal format**
2. Show only the **first few lines** of that hex output

Let's break it:

- xxd → Converts a file into **hex dump format**
- evidence.txt → Your text file
- | → Pipe (sends output to another command)
- head → Shows only the first 10 lines (default)

📌 So this command shows the **hexadecimal representation of the beginning of evidence.txt**

🔷 **xxd evidence.img | head**

This is the same idea, but:

- evidence.img is usually a **disk image file** (for example created using dd)
- This lets you see the **raw binary data in hex format**

📌 In digital forensics, this is used to:

- Check file headers (file signatures)
- Verify image content
- Inspect hidden or suspicious data
- Confirm if the image was created correctly
- Analyze raw disk data

Why This Is Important in Forensics

When you create a disk image using dd, you use xxd to:

- Inspect the image at **bit-level**
- Detect hidden strings
- Find malware signatures
- Check file system structures

2. Chech Size of Files

```
Code

ls -l evidence.txt
ls -l evidence.img
```

🔷 **ls -l evidence.txt**

This command is used to:

- Show **detailed information** about the file evidence.txt

🔷 **ls -l evidence.img**

Same command, but for the disk image file.

```
┌──(kali㉿kali)-[~/Semester_6/Lab1_Advanced_Disk_Imaging]
└─$ ls -l evidence.txt
ls -l evidence.img
-rw-rw-r-- 1 kali kali 31 Mar  2 02:39 evidence.txt
-rw-rw-r-- 1 kali kali 31 Mar  2 02:40 evidence.img

┌──(kali㉿kali)-[~/Semester_6/Lab1_Advanced_Disk_Imaging]
└─$
```

Result:

The original file (31 bytes) and the disk image (31 bytes) have identical sizes, confirming that the imaging process was successful and no data was lost during acquisition.

3. Check File Type

```bash
file evidence.txt
file evidence.img
```

This shows how Linux identifies the file.

```
┌──(kali㉿kali)-[~/Semester_6/Lab1_Advanced_Disk_Imaging]
└─$ file evidence.txt
file evidence.img
evidence.txt: ASCII text
evidence.img: ASCII text

┌──(kali㉿kali)-[~/Semester_6/Lab1_Advanced_Disk_Imaging]
└─$
```

That proves structure is identical.

4. Simulate Tampering

```bash
echo "Hacked" >> evidence.txt
```

```
┌──(kali㉿kali)-[~/Semester_6/Lab1_Advanced_Disk_Imaging]
└─$ echo "Hacked" >> evidence.txt

┌──(kali㉿kali)-[~/Semester_6/Lab1_Advanced_Disk_Imaging]
└─$ ls
evidence.img  evidence.txt  md5_hash.txt  sha256_hash.txt

┌──(kali㉿kali)-[~/Semester_6/Lab1_Advanced_Disk_Imaging]
└─$ ca evidence.txt
ca: command not found

┌──(kali㉿kali)-[~/Semester_6/Lab1_Advanced_Disk_Imaging]
└─$ cat evidence.txt
Confidential forensic evidence
Hacked

┌──(kali㉿kali)-[~/Semester_6/Lab1_Advanced_Disk_Imaging]
└─$
```

```
┌──(kali㉿kali)-[~/Semester_6/Lab1_Advanced_Disk_Imaging]
└─$ sha256sum evidence.txt
sha256sum evidence.img
cf3a368303b77e9491c31f80c965d6750eae6dee193a239213a32c1b5ff9f6d5  evidence.txt
516fc8a3fedace63daa40cf6841ea2c1c72cfdfd6be8c10819fef2d08663a45d  evidence.img

┌──(kali㉿kali)-[~/Semester_6/Lab1_Advanced_Disk_Imaging]
└─$
```

So now Hash is differeny because we change in file file.

5. Check difference location

```bash
cmp -l evidence.txt evidence.img
```

This shows exact byte number where change happened.

```
┌──(kali㉿kali)-[~/Semester_6/Lab1_Advanced_Disk_Imaging]
└─$ cmp -l evidence.txt evidence.img
cmp: EOF on 'evidence.img' after byte 31

┌──(kali㉿kali)-[~/Semester_6/Lab1_Advanced_Disk_Imaging]
└─$
```

```
BS-CyberSecurity-6th-Semester/
├── Digital-Forensics-Lab/
│   ├── README.md                          # Main lab overview
│   ├── Lab-01-Disk-Imaging-using-dd/
│   │   ├── README.md                       # Lab 1 specific README
│   │   ├── Lab-Report.md                    # Your lab documentation
│   │   ├── images/                          # Folder for screenshots
│   │   │   ├── image1.png
│   │   │   ├── image2.png
│   │   │   ├── image3.png
│   │   │   ├── image4.png
│   │   │   └── ... (all your 19 images)
│   │   ├── evidence/                         # Evidence files
│   │   │   ├── evidence.txt
│   │   │   └── evidence.img
│   │   └── scripts/                          # Helper scripts
│   │       └── forensic-commands.sh
│   ├── Lab-02-xxx/
│   └── Lab-03-xxx/
```